Customer App Module Flow Guide

Enatega Super App Customer Application

Modules covered: RideSharing and Deliveries

Document type: Client Guide

Prepared from current codebase snapshot: June 19, 2026

This guide explains the customer journey, module behavior, and key operational considerations for the RideSharing and Deliveries experiences within the unified customer app.

Customer App Module Flow Guide

Project: Enatega Super App Customer Application

Modules Covered: RideSharing and Deliveries

Document Type: Client Guide

Prepared From Current Codebase Snapshot: June 19, 2026

1. Purpose of This Document

This document explains how the customer application works at a module level for the RideSharing and Deliveries experiences. It is intended for client stakeholders who need a clear view of the customer journey, the responsibilities of each module, and the important operational points that affect rollout, support, and day-to-day usage.

The application is a single customer app that contains multiple service capabilities under one shared platform. In the current scope, the two major customer modules are:

? RideSharing

? Deliveries

Both modules run inside one shared mobile application and use common platform services such as authentication, profile management, saved addresses, wallet-related flows, notifications, localization, and support.

2. Overall Customer App Architecture

At a high level, the customer app works in three layers:

1. Shared App Layer

This layer handles the global customer experience, including app entry, authentication, profile data, saved addresses, theme, localization, notifications, and navigation between mini-apps.

2. RideSharing Module

This module manages ride booking, courier booking, fare estimation, live driver matching, active trip tracking, ride chat, safety actions, ride history, reservations, and post-ride feedback.

3. Deliveries Module

This module manages delivery-mode setup, store discovery, search, product browsing, cart, checkout, order placement, order tracking, rider chat, support, wallet screens, and profile-related flows.

3. Shared Platform Features Used by Both Modules

The two modules are separate in business flow, but they rely on the same customer account and common utility layers.

? Authentication is shared across the entire app.
? A single customer profile is used across modules.
? Saved addresses are reusable across supported flows.
? Notifications are handled centrally and surfaced in-app.
? Localization is already structured for English and French.
? Theme and branding are centrally managed.
? Shared support and settings flows are available at the app level.
? Shared navigation allows the user to move between the home app selector and enabled service modules.

For the client, this means the customer does not need separate apps or separate accounts for RideSharing and Deliveries. The experience is designed as one consolidated customer platform.

4. RideSharing Module Flow

4.1 RideSharing Entry and Initialization

When the customer enters the RideSharing module, the app initializes RideSharing-specific configuration before the full journey continues. This configuration includes operational values such as active currency and emergency contact information.

The RideSharing home experience is not just a static landing page. It behaves like a state-driven controller that can show one of the following views depending on the customer?s current status:

? Standard ride booking view
? Finding ride view while a request is being matched
? Active ride view when a ride is already in progress
? Completed ride feedback sheet after the trip ends

This design helps the app resume the correct customer state even if the user returns to the module in the middle of a journey.

4.2 RideSharing Main Customer Journey

The standard RideSharing booking flow follows this sequence:

1. Customer opens RideSharing.
2. Customer selects the ride intent or service type.
3. Customer searches for pickup and destination addresses.
4. Customer reaches the fare estimation screen.
5. Customer selects a ride option and payment method.
6. Customer confirms the booking request.
7. The app creates the ride request and begins live driver matching.
8. The customer either accepts a driver bid or continues searching.
9. Once matched, the app moves into active ride tracking.
10. After trip completion, the customer can rate the experience and review ride details.

4.3 Ride Type and Service Selection

The first RideSharing view allows the customer to select the intended service from available ride types returned by backend configuration. This means the visible ride options are not completely hardcoded. They are designed to reflect what the platform makes available.

From the current flow, RideSharing supports:

? Standard ride booking
? Courier-oriented ride flow
? Scheduled ride flow
? Hourly fare flow where applicable

The module also remembers the customer?s recently used addresses and preferred payment method to reduce booking friction.

4.4 Address Search and Trip Setup

After selecting a service, the customer moves to address search. In this step, the app supports:

? Pickup location selection
? Drop-off location selection
? Multiple stops where supported
? Recent address reuse
? Map-assisted location choice

This stage prepares the trip structure that will be used for route drawing, fare estimation, and request creation.

4.5 Fare Estimate and Booking Confirmation

Once the route is known, the app requests:

? Route path data for trip visualization
? Fare estimates for available ride types

On the estimate screen, the customer can:

? Review pickup and drop-off summary
? View available ride categories
? Select a payment method
? Schedule the ride for later
? Adjust the offer fare in supported ride modes
? Enter courier details when the journey is courier-related

This is the main decision screen before a ride request is sent.

4.6 Live Matching and Bid-Based Search

After confirmation, the RideSharing module enters a live matching phase. This is one of the most important parts of the flow.

The app creates the ride request through the API, then uses socket-based real-time communication to:

? Broadcast the request to nearby drivers
? Receive incoming bids
? Update search state without requiring a screen refresh
? Allow the customer to accept or reject driver bids
? Allow continued searching if no suitable match is found
? Allow fare increase in supported matching flows

This means the module is not only API-driven. It also depends on a real-time socket layer for driver discovery and ride progression.

4.7 Active Ride Flow

After a driver is accepted, the module transitions into active ride management. The home screen can automatically switch into an active ride overlay if a live trip already exists.

During an active ride, the module supports:

? Real-time ride status updates
? Trip map and route visibility
? Driver profile access
? Rider-to-driver chat
? Safety information and emergency actions
? Support chat access

This active-state overlay is important for business continuity because the customer can reopen the app and return directly to the current ride state.

4.8 Post-Ride Experience

After trip completion, the customer journey does not end immediately. The module includes:

? Trip completion feedback and rating flow
? Ride history
? Reservation list and reservation detail views
? Notification screens
? Wallet-related screens for ride payments and card handling

This creates a full service loop from booking through post-service engagement.

4.9 RideSharing Operational Notes for Client

? RideSharing depends on both backend APIs and real-time socket connectivity.
? Currency and emergency contact data are initialized through module configuration.
? Payment handling is customer-selectable and currently includes cash, wallet, and card-oriented flows depending on availability.
? The home view is state-aware, which allows ride recovery if the customer reopens the app during search or during an active ride.
? Scheduled and courier scenarios are supported as dedicated variants of the same RideSharing module rather than as separate apps.

5. Deliveries Module Flow

5.1 Deliveries Entry and Initialization

When the customer enters Deliveries, the app first initializes delivery-specific platform configuration. This includes:

? Platform type

- Delivery mode
- App settings
- Currency data

This configuration is cached locally so the app can recover faster on the next launch, while still refreshing from the backend when needed.

An important business detail is that the Deliveries module is configuration-driven. Based on the platform type, the customer is routed into one of three delivery operating models:

- Single Vendor
- Multi Vendor
- Chain

This allows the same customer app to support multiple delivery business structures without requiring separate customer applications.

5.2 Deliveries Module Structure

The Deliveries module contains three storefront experiences under one shared delivery umbrella.

Single Vendor

This mode is used when the customer is interacting with one dedicated vendor structure. The module presents:

- Home tab
- Search tab
- Orders tab
- Profile tab
- Delivery details flow

Multi Vendor

This mode is used when the customer browses multiple vendors and store types in one marketplace model. The module presents:

- Home tab
- Search tab
- Orders tab
- Profile tab
- Favourites

? Store details
? Category and discovery extensions
? Map-based and see-all browsing flows

Chain

This mode is used for chain-style catalog structures. In addition to the standard tabs, the flow supports menu-template-based browsing so a chain can switch categories or structured menus cleanly.

5.3 Deliveries Main Customer Journey

The standard Deliveries journey follows this sequence:

1. Customer opens Deliveries.
2. Delivery mode is resolved from configuration.
3. Customer selects or confirms an address.
4. Customer browses stores, categories, offers, or search results.
5. Customer opens a store or product detail.
6. Customer adds items to cart.
7. Customer reviews cart and resolves any store conflict rules.
8. Customer proceeds to checkout.
9. Customer selects fulfillment, payment, notes, schedule, and tip.
10. Customer places the order.
11. Customer tracks order status in real time.
12. Customer can chat with the rider where applicable.
13. Customer reviews order details and rates the order after completion.

5.4 Address Selection as a Core Deliveries Dependency

Address selection is central to the Deliveries experience. The delivery home flows are built around the customer?s selected address because store availability, offers, and checkout depend on fulfillment context.

Across delivery modes, the customer can:

? Select from saved addresses
? Add a new saved address
? Use current location
? Choose location on map

This makes address management one of the first operational dependencies the client should validate during testing.

5.5 Discovery and Browsing Flow

The browsing experience changes slightly by delivery mode, but the core pattern is consistent.

In Multi Vendor, the home and search flows emphasize:

? Shop types
? Nearby stores
? Top brands
? Deals
? Order again
? Marketplace search and discovery

In Single Vendor, the home flow is more focused on:

? Promotional banners
? Vendor categories
? Vendor deals

In Chain mode, the home flow includes:

? Promotional banners
? Category sections
? Deals
? Menu template selection for chain-specific catalog organization

This means each delivery mode has a different storefront presentation, but all of them lead into a shared purchase pipeline.

5.6 Product, Cart, and Checkout Flow

After discovery, the customer can open product details, add items to cart, and move into checkout.

The cart and checkout layers are among the most business-critical parts of the Deliveries module. The current implementation supports:

? Product detail-driven add-to-cart actions
? Cart validation rules
? Store conflict handling when products come from incompatible stores

- Checkout preview before final submission
- Delivery or pickup-style order type logic where configured
- Customer notes for restaurant and courier
- Leave-at-door handling
- Delivery scheduling
- Rider tips
- Coupon application
- Multiple payment paths including codified support for card and Stripe-related flows where enabled

The checkout flow also refreshes the cart and order queries after successful placement so the customer immediately sees the updated order state.

5.7 Order Placement and Post-Checkout Flow

After checkout, the app sends the order to backend services and routes the customer into post-order management.

The module then supports:

- Orders list
- Order details
- Live order tracking
- Rider chat
- Rate order flow

This creates a complete commerce loop from browse to fulfillment follow-up.

5.8 Real-Time Delivery Features

The Deliveries module is not purely static after order placement. It includes live operational features such as:

- Order tracking status updates
- Rider chat
- Delivery support communication

As with RideSharing, this means backend services and real-time connectivity are both important for a complete customer experience.

5.9 Deliveries Operational Notes for Client

- Deliveries is configuration-driven and can launch into Single Vendor, Multi Vendor, or Chain mode based on backend settings.
- Cached bootstrap configuration improves reopen speed while still allowing remote refresh.
- Address accuracy is critical because storefront visibility, delivery coverage, and checkout depend on it.
- Checkout supports several operational modifiers such as schedule, tips, notes, and payment choices.
- The order journey continues after placement through tracking, chat, support, and rating.

6. Shared Support, Wallet, Profile, and Settings Flows

Although RideSharing and Deliveries have different commercial flows, both modules rely on common customer-account capabilities.

These include:

- Profile viewing and editing
- Address management
- Notification screens
- Wallet and saved card flows
- Language settings
- Appearance or theme settings
- Privacy and terms screens
- Password and account management
- Support conversations and ticketing flows

For the client, these shared layers reduce duplication and make the app feel like one platform rather than multiple disconnected products.

7. What Is Important for the Client to Know

The following points are especially important for business review, user acceptance testing, and launch planning.

7.1 The Customer App Is a Unified Platform

RideSharing and Deliveries are separate modules, but they share one customer identity, one app shell, and several common utility layers. This simplifies customer onboarding and increases cross-service convenience.

7.2 Several Behaviors Are Controlled by Backend Configuration

The app is not entirely fixed at build time. Important behavior can be shaped by backend responses, including:

? Active delivery mode
? Currency
? Emergency contact data
? Branding-related app settings
? Maintenance messaging
? Available service content

This gives the client flexibility, but it also means backend configuration must be validated carefully before release.

7.3 Real-Time Connectivity Is Essential for Service Quality

Both modules rely on real-time features.

? RideSharing uses real-time matching and trip-state synchronization.
? Deliveries uses real-time order tracking and chat-style communication.

If real-time infrastructure is degraded, core user journeys will still partially exist, but the service quality and responsiveness will be affected.

7.4 Address Data Quality Has Direct Business Impact

Saved address accuracy is especially important for Deliveries and still important for RideSharing.

? It affects store availability.
? It affects checkout readiness.
? It affects route and fare calculations.
? It affects pickup and drop-off correctness.

7.5 Payment Experience Depends on the Enabled Business Setup

The app already supports multiple payment-related paths, but the exact operational experience depends on backend-enabled methods and business policy. Client-side testing should therefore validate:

? Cash or COD paths
? Wallet-related flows
? Saved card behavior

? Stripe or card-based payment paths where enabled

7.6 Testing Should Cover Resume and Recovery Scenarios

Because the app maintains active ride and active order states, testing should not only cover fresh bookings and fresh orders. It should also cover:

? Reopening the app during active ride search
? Reopening the app during an active ride
? Reopening the app after order placement
? Reopening the app during order tracking
? Notification-driven entry into active flows

These recovery scenarios are important for real-world customer reliability.

8. Recommended Client Review Checklist

? Confirm which delivery operating mode is intended for launch.
? Validate backend configuration for currency, branding, and maintenance messaging.
? Test saved address selection, current location, and map-based address flows.
? Test RideSharing request creation, driver matching, active trip state, and trip completion.
? Test Deliveries browsing, cart, checkout, order placement, tracking, and rating.
? Test payment methods that will be active in production.
? Test support, chat, and notifications with real data.
? Test resume behavior after app close or backgrounding.
? Test English and French content completeness.

9. Conclusion

The current customer app is designed as a modular service platform rather than a single-purpose application. RideSharing and Deliveries are implemented as distinct customer journeys, but they operate on top of one shared customer foundation.

From a client perspective, the most important strengths of the current architecture are:

? One customer app for multiple services
? Configurable delivery business models
? Real-time operational support for rides and orders
? Shared account, address, profile, and support layers
? End-to-end flows from discovery or booking through completion and follow-up

This makes the app suitable for a multi-service customer experience, while still allowing each business module to keep its own operational logic and customer journey.